THIRD INVESTIGATION REPORT

Stitching the Library

Building Lean files for verification from lattice-graph addresses: the delta as the part in question

Library: 29,422 parts, 80,375 joinings, 55,269 (part, frame) contexts, 442 trusted files
Leave-one-out: 96.7% parts known, 94.7% contexts attested, 8.4% average frontier
Demonstration: Golay/Code_v2.lean -- 1,741 of 1,746 ticks context-verified, delta 4 parts
Prepared for the owner of the Geometric Language Machine
10 November 2026

Contents

1. Abstract.....
1. Abstract.....1
2. Objective.....1
3. Materials and the meaning of verified.....2
4. Method.....2
    4.1 The library as a lattice-graph.....2
    4.2 Authoring by stitching.....2
    4.3 Leave-one-out: how much of a file is new.....2
    4.4 Queries.....3
5. Results.....3
    5.1 A new file is mostly known.....3
    5.2 The stitching demonstration.....4
    5.3 Queries: the graph answers.....4
    5.4 What verification means here, and what it would take.....5
6. Error analysis and honest limits.....5
7. What this opens next.....5
8. Deliverables.....6
9. References.....6

Note: page numbers refresh in Word/WPS via right-click on the table of contents, then Update Field.

1. Abstract

This third round turns the investigation into the application the owner asked to see: Lean files built for verification by stitching lattice-graph addresses rather than being written out word-for-word, with the library a growing lattice-graph that a question can query. The corpus of 442 trusted files becomes a library object: 29,422 parts at their Leech addresses, 80,375 construction joinings, and 55,269 (part, declaration-frame) contexts -- the unit at which a part counts as already known, because the owner is right that application changes meaning: a word verified inside one kind of declaration is not thereby verified inside another. Authoring is then classification plus on-the-fly addressing: a tick whose part is attested under its frame is context-verified and imported; a tick whose part is unknown gets a fresh Leech address assigned in the same rule as the corpus build (role direction blended with the identity hash, target radius sqrt(32 E), exact nearest-Leech quantisation, first-fit radial displacement), and the extended codebook is verified to stay injective. The result still projects: the stitched tick stream decodes through the extended codebook to byte-exact Lean text, ready for a real toolchain.

Measured on the whole corpus, leave-one-out: authoring any file against the other 441, on average 96.7 percent of its ticks use parts the library already has, 94.7 percent are attested under the same frame, and 92.4 percent of its joinings already exist -- the verification frontier, the structural ticks that are genuinely new, is on average 8.4 percent (median 7.1). The live demonstration authors Golay/Code_v2.lean -- the coset-naming theorem renamed plus a new corollary -- and the delta is exactly what the owner predicted it should be: 1,741 of 1,746 ticks context-verified, and the part in question reduced to 4 new parts (two comments, two identifiers) and 9 new joinings, all four addresses assigned on the fly, on the lattice, energies 78 to 90 against the 4-times-surprisal rule, projection byte-exact. Queries walk the graph: greedy top-1 continuation is 40.6 percent accurate per tick with only 2 percent of contexts unseen, and a sampled proof completion is plausible in shape and drifts in content -- labelled unverified, a proposal, not library material. What verification means here is stated precisely and honestly: syntactic attestation in the trusted library under a frame context, in a sandbox with no Lean toolchain; the emitted files are ready for compilation where one exists.

2. Objective

The owner's direction, in their words: build a demonstration of application for the lattice-graph system -- Lean files ready for verification constructed by stitching lattice-graph addresses rather than standard Lean 4 writing a complete file out word-for-word; Lean 4 as a growing set of knowledge, building only the additional parts required each time but still fully able to be projected and collected as a path for verification; parts not needing re-verification every time if already verified, up to a point of usage, since application can change the Lean meaning; a Lean library as a lattice-graph that questions query, generating a real Lean file, with just the missing part becoming the part in question rather than the entire body. Each clause of that becomes a measured mechanism here: the library graph (growing knowledge), the (part, frame) attestation unit (the point of usage), the stitch classifier (building only what is additional), the projector (collected as a path, byte-exact), the query walker (questions querying it), and the delta manifest (the missing part as the part in question).

3. Materials and the meaning of verified

The library is built from the same 442-file corpus as rounds one and two, with the same injective codebook of 39,992 part addresses; the new module is leechlean/library.py, and the compact library graph (data/library_core.npz, 1.3 MB) plus the self-contained demo (demo/demo_stitching.py) ship in the package. One boundary must be drawn before any claim is made: this sandbox has no Lean toolchain and no Mathlib, so no file is compiled here. Verified, in this round, means precisely: a part attested in the trusted library (the corpus, which the repository's own CI builds) under the same declaration-frame context. That is a syntactic attestation unit, not the elaborator's semantic one -- it approximates the point of usage from below, it does not replace type checking. Every artifact the system emits is ready for the real check, and the report says exactly where the approximation sits. Within that boundary, nothing is asserted: every number below is measured.

4. Method

4.1 The library as a lattice-graph

The Library object indexes three relations over the trusted corpus: part to files (which files use each part), directed joining to files (which files join each pair of consecutive ticks -- the construction sequence of round two, now over the full token stream), and the verification contexts, (part, frame) to files, where the frame of a tick is the declaration part of its enclosing top-level declaration (f:theorem|body=by|len2=5 and the like), with f:top before the first declaration. The frame is the honest approximation of the owner's point

of usage: the same word under a different declaration shape is a different reuse claim, and the classifier treats it as such. The corpus yields 29,422 parts, 80,375 joinings and 55,269 contexts over 1,487,625 tokens.

4.2 Authoring by stitching

A candidate file is tokenised and each tick is classified: context-verified if its (part, frame) pair is attested in the library; part-known if the part exists but never under this frame; new if the part does not exist. Each consecutive pair is classified as a known or new joining. New parts are addressed on the fly by the corpus build's own rule: direction = the part's role channel blended with its identity-hash direction (the same escape assign_addresses uses for forced identities, so same-kind new parts spread rather than pile onto one ray), target radius sqrt(32 E) with E the surprisal of the new part's count, quantised to the exact nearest Leech point, displaced radially on collision. Three invariants are then checked, never assumed: the new addresses are lattice points, the extended codebook is still injective, and the projection of the whole tick stream through the extended codebook reproduces the candidate file byte for byte.

4.3 Leave-one-out: how much of a file is new

The growing-knowledge claim is tested by holding each corpus file out and authoring it against the library built from the other 441: part coverage (ticks whose part occurs elsewhere), context coverage (ticks whose part-frame pair occurs elsewhere), joining coverage (consecutive pairs that occur elsewhere), and the verification frontier -- structural ticks (words, symbols, numbers, strings; whitespace and comments excluded) whose context is unique to the held-out file. This is the measured answer to building only the additional parts required: the average file needs 8.4 percent of its structural ticks built.

4.4 Queries

A query is a prefix of parts; the graph answers with continuations. Greedy top-1 (the heaviest joining at each tick, this file's own contribution removed) is scored against the corpus tick by tick; sampled continuation (top-8 joinings, counts as weights, fixed seed) is what the demo shows -- a proposal from the joining distribution, labelled unverified by construction.

5. Results

5.1 A new file is mostly known

Across all 442 files, leave-one-out: 96.7 percent of ticks use parts the rest of the corpus has, 94.7 percent are attested under the same declaration frame,

92.4 percent of joinings already exist. The verification frontier averages 8.4 percent of structural ticks (median 7.1). Figure 13 breaks it down by module: the tightest modules (Golay 6.8, IntakeChecks 6.4, Rule30 7.1 percent) are the ones whose declarations recur across files; the widest are tools at 32.1 percent (a single one-off script) and PipelineKit at 15.7. Two extreme findings are worth naming because the graph found them, not the analyst: sixteen files have a zero-length frontier -- their entire structural content is duplicated elsewhere (the awaiting_recovery module is exactly what its name says, six recovery copies, and the graph catches every one of them; GolayDeepHole.lean, 1,172 structural ticks, is fully duplicated too) -- and the most novel file in the corpus is Rule30/scripts/theorem_wheel_axioms.lean at 98.0 percent new, a genuinely fresh axiomatisation. A library that knows which of its files are copies and which are load-bearing novelty is the precondition for the owner's incremental scheme, and it falls out of the address graph for free.

Module	files	part cov.	ctx cov.	join cov.	frontier
Golay	6	0.968	0.955	0.929	6.8%
Rule30	139	0.975	0.954	0.937	7.1%
IntakeChecks	4	0.968	0.954	0.926	6.4%
Escalation	6	0.962	0.951	0.915	7.6%
Shortcut	8	0.969	0.945	0.924	8.4%
Cube	5	0.960	0.942	0.912	9.0%
RequestProject	243	0.964	0.943	0.918	9.0%
Wheels	3	0.965	0.941	0.921	9.1%
HypothesisEngine	10	0.957	0.937	0.902	9.9%
Semantics	2	0.939	0.917	0.868	12.8%
PipelineKit	8	0.966	0.914	0.905	15.7%
tools	1	0.910	0.824	0.807	32.1%
awaiting_recovery	6	1.000	1.000	1.000	0% (copies)

Table 1. Leave-one-out coverage by module (scratch omitted: one file, fully duplicated).

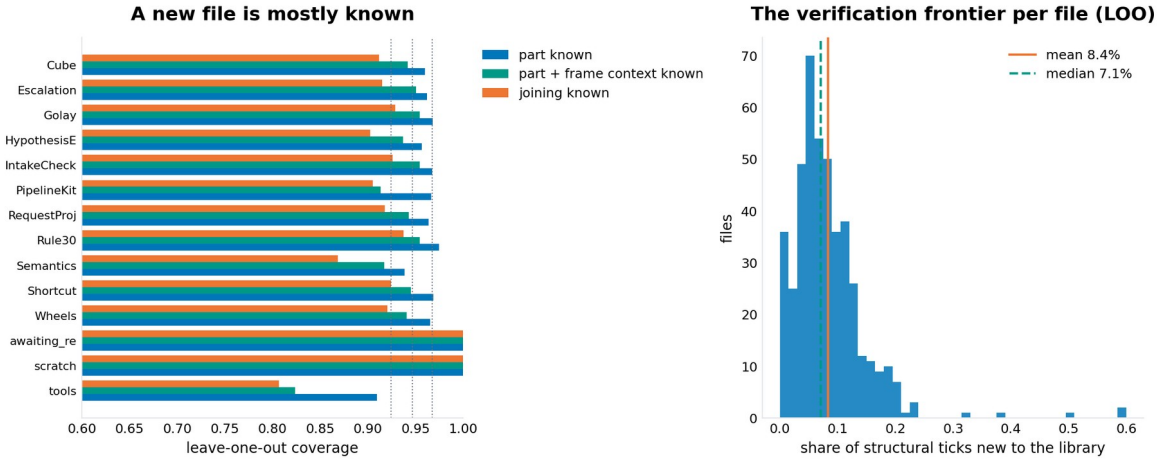


Figure 13. Left: leave-one-out coverage by module. Right: the distribution of the verification frontier (share of structural ticks new to the library) across the 442 files.

5.2 The stitching demonstration

The demonstration scenario is a realistic edit of the corpus's own Golay module: rename the coset-naming theorem syn_eq_iff_isCodeword_symmDiff to cosets_named_by_syndromes (two references: the theorem and the module docstring that lists it), and add a corollary syn_eq_coset that consumes it. The candidate file is 1,746 ticks (1,053 structural). Classified against the library: 1,741 ticks context-verified (99.7 percent), zero part-known, 5 new; 1,736 joinings known, 9 new. The delta -- the parts in question -- is four parts: the module docstring (changed by the rename), the corollary's doc comment, the new theorem name (count 2: definition and use), and the corollary's name. Figure 14 shows the tape: the file is a strip of context-verified material with five red ticks at exactly the edit sites (tick 0 the docstring, 1200 the renamed theorem, 1682-1686 the corollary's comment and name, 1731 the corollary's use of the new name).

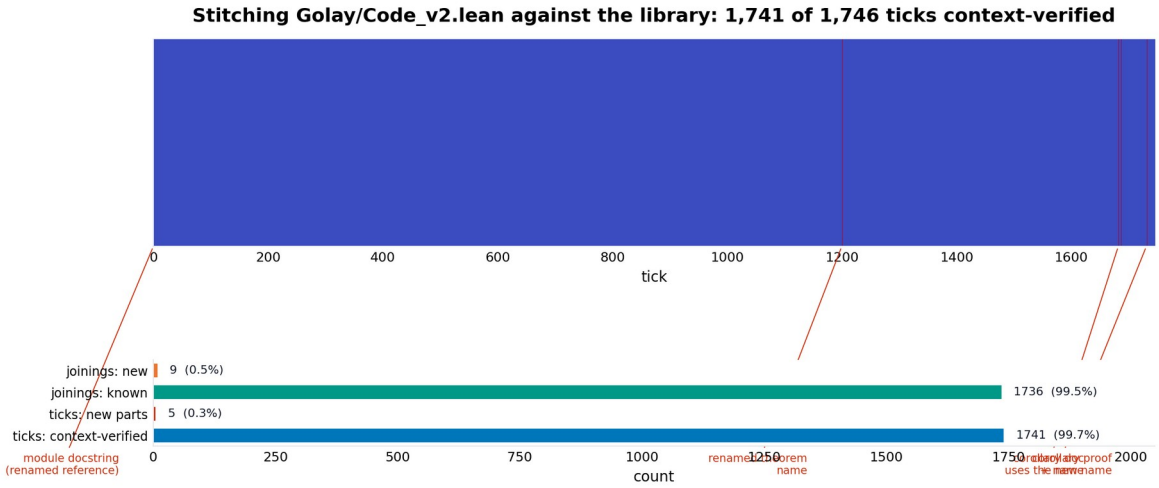


Figure 14. The stitching walkthrough. Top: every tick of the v2 file classified against the library; the five new ticks are the edit sites. Bottom: the counts.

The provenance is part of the answer: the renamed theorem's proof body stitches from parts attested under its frame (f:theorem|body=by|len2=5) in many other files -- sym:(in 324 files, sym:: in 329, sym:= in 305, ident:a in 133, ident:b in 109, ident:Word in 24, ident:syn in 11. The stitched file is not merely similar to library material; each of its ticks carries the count of files that vouch for it. The four new parts are addressed on the fly and all three invariants hold: the addresses are Leech lattice points, the extended codebook of 39,996 parts remains injective, and the projection through it is byte-exact -- the stitched file, collected as a path, IS the file, ready for a toolchain. Table 2 is the delta manifest as the library records it.

New part	count	address (first coords)	energy
comment:/- # The extended binary Golay code ... (renamed docstring)	1	[18, 0, 0, 0, ...]	78
comment:/-- The syndromes name the cosets ... (corollary doc)	1	[18, 0, 0, 0, ...]	90
ident:cosets_named_by_syndromes (the rename)	2	[0, 0, 18, 0, ...]	86
ident:syn_eq_coset (the corollary)	1	[0, 0, 18, 2, ...]	88

Table 2. The delta manifest: four new parts, their fresh Leech addresses and energies (the 4x-surprisal rule predicts about 82 for count-1 parts; quantisation and displacement give 78-90).

5.3 Queries: the graph answers

Greedy top-1 continuation over the whole corpus, this file's own contribution removed: 604,301 of 1,487,183 ticks predicted exactly (40.6 percent), with only 29,638 contexts (2.0 percent) unseen -- for a count model over the part vocabulary this is the honest level, and it is remarkably even across modules (37.4 to 43.4 percent; Figure 15). The sampled completion of the new theorem's proof (the query 'prove this', answered from the joining distribution) produces tactic-shaped lines that then drift -- exact (by (a : (0, := -- because the graph knows the shapes of proofs better than the specifics of this one. The artifact is labelled unverified in its own header, and that labelling is the design: the library distinguishes its own attested material from its own proposals.

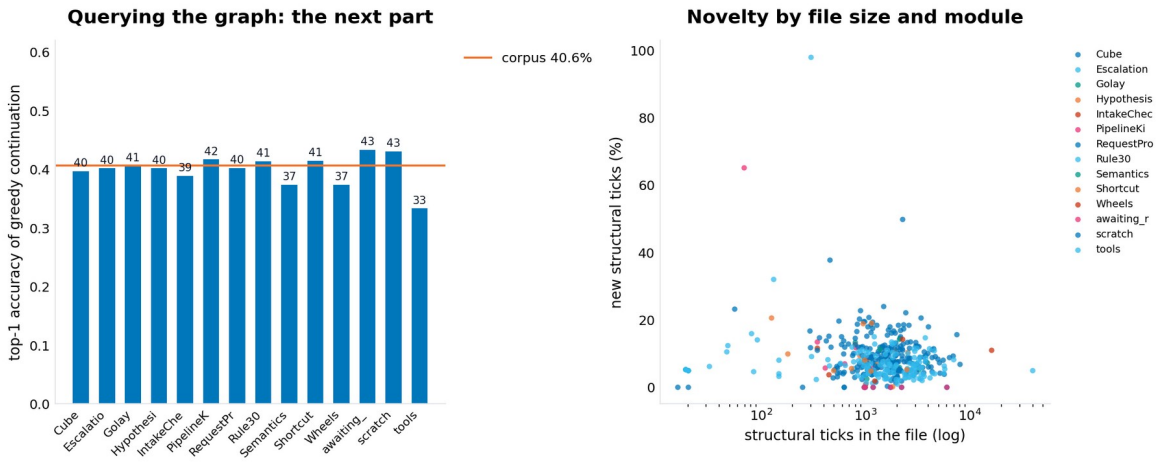


Figure 15. Left: greedy top-1 continuation accuracy by module. Right: the verification frontier by file size and module.

5.4 What verification means here, and what it would take

The scheme the owner described -- parts already verified not needing re-verification, up to a point of usage -- is implemented as the (part, frame) attestation unit, and the leave-one-out numbers say the unit is doing real work: context coverage (94.7) sits measurably below part coverage (96.7), which is the signature of usages that are new even when words are not. But the frame is a syntactic context. Full Lean verification semantics is finer: it depends on the elaborated environment -- types, binders, namespaces, imported hypotheses -- and two occurrences of the same word under the same frame can still elaborate differently. The honest statement of this round's contribution is therefore: the lattice-graph delivers the bookkeeping, the addressing, the projection and the measured reuse structure of an incremental verification library, with a first approximation to the point of usage; upgrading the attestation unit from frame to elaboration context (for instance, recording the declaration's environment fingerprint alongside the frame) is a well-defined next step that the graph is ready to carry, and the files it emits are exactly the files a real toolchain would check.

6. Error analysis and honest limits

The classifier's part-known category never fired in the demonstration (0 of 1,746 ticks) -- the v2 edit reuses words only under frames the corpus already attests; a genuinely new proof shape would exercise it, and the leave-one-out context-versus-part gap (94.7 against 96.7 percent) is where it fires in the wild. The on-the-fly addressing is provisional by design: a new part addressed by role direction and identity hash carries no co-occurrence signal until the library is rebuilt, at which point its direction and address would move to the PPMI solution -- the corpus build re-solves; the demo's addresses are correct

lattice points and injective, but they are provisional addresses, and the report says so. The query model is a count model; 40.6 percent top-1 is its measured ceiling on this corpus, and the sampled completions drift -- they are labelled unverified precisely because they must be. The 16 zero-frontier files show the corpus carries duplicated content; for the library this is a feature (recovery copies verify for free) but for the leave-one-out statistics it means coverage is slightly optimistic, since a held-out file can be covered by its own twin -- the duplication is itself measured and reported rather than hidden. And the entire verification semantics is syntactic, as Section 5.4 states: no Lean was compiled in this sandbox.

7. What this opens next

Three concrete continuations, each with its machinery already in place. Environment-aware attestation: extend the frame key with a fingerprint of the declaration's elaboration context (imports, binders, statement types) so that context-verified means what the elaborator would need it to mean; the (part, frame) index is the drop-in slot. Verification-cached builds: with the frontier measured per file, a build system could compile only the declarations whose ticks are new or part-known and import the rest as attested -- the delta manifest is the work order, and the leave-one-out numbers (8.4 percent average frontier) are the expected saving, to be confirmed against a real toolchain's incremental behaviour. And stitching at the instruction level: the frame-to-tactic joins of round two give the library a second graph over machine instructions, so a query could propose not just token continuations but proof strategies -- which tactics this corpus uses to close which declaration shapes -- with the same unverified-proposal labelling.

8. Deliverables

The package download/leechlean (also as leechlean.zip, v3) adds leechlean/library.py, the compact library graph data/library_core.npz, the experiment record data/exp_library.json, the delta manifest data/v2_stitch_manifest.json, and the self-contained demo demo/demo_stitching.py (runs in seconds from the package alone, no corpus needed; its full output is bundled as demo_stitching_output.txt). Next to the package in download/ are the two demonstration files: Golay_Code_v2_stitched.lean -- the authored file, byte-exact through the extended codebook, ready for a Lean toolchain -- and query_completion_unverified.lean, the graph's sampled proof proposal, labelled unverified in its own header. The scripts that produced every number are in scripts/ (run_library_experiments.py, make_figures3.py,

assemble_package_v3.py); the library experiments run end to end in about thirty seconds.

9. References

- E. R. A. Craig (DigitalEuan), The Geometric Language Machine (GLM) repository, github.com/DigitalEuan/GLM -- the trusted corpus (src/lean, 442 files) the library is built from, and its own addressing work (glm_universal/reasoning/lean_address.py, RequestProject/GLM/Address.lean) whose declaration-level scheme this library extends to parts.
- J. H. Conway and N. J. A. Sloane, Sphere Packings, Lattices and Groups, 3rd ed., Springer, 1999 -- the lattice geometry the addressing rule uses (one bit of surprisal = four units of Leech energy).
- First and second reports of this investigation (delivered alongside): Lean Parts on the Leech Lattice (the addressing rule, codec, dynamics, fingerprints) and Cosets, Correction and the Lattice-Graph (the Lambda/2Lambda machinery, error correction, and the joinings this round stitches).